

DEADMAN SWITCH VAULT

SECURITY AUDIT REPORT

AUDITED BY

ZUHAIBMOHD



Deadman Security Review Report

Introduction

A time-boxed security review of the Deadman (deadman switch vault) program was conducted by **zuhaibmohd**, with a focus on the security aspects of the smart contracts.

Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource, and expertise-bound effort where I attempt to find as many vulnerabilities as possible.

⚠️ Note

We cannot guarantee 100% security after the review or even that the review will uncover any vulnerabilities.

It is recommended to conduct subsequent reviews, run bug bounty programs, and enable on-chain monitoring to ensure long-term security.

About zuhaibmohd

zuhaibmohd is an independent smart contract security researcher.

Check out his [previous work](#) or connect on X [@zuhaib44](#).

About Deadman

Deadman is an **Anchor program** on Solana implementing a **deadman switch vault**. An owner creates a vault with a beneficiary, deposits tokens, and must periodically call `ping` within a configured `interval` plus `grace_period`. If the owner stops pinging (e.g. incapacity or death), the vault is considered expired and the **beneficiary** can call `withdraw` to claim all tokens and close the vault. The contract supports: `initialize`, `deposit`, `ping`, `owner_withdraw`, `withdraw` (beneficiary), `cancel_vault`, `update_beneficiary`, and `update_interval`.

Severity Classification

Severity	Likelihood →	High	Medium	Low
Impact ↓				
High	Critical	High	Medium	
Medium	High	Medium	Low	
Low	Medium	Low	Low	Low

- **Impact:** The potential technical, economic, and reputational damage from a successful exploit.
- **Likelihood:** The probability that a given vulnerability will be discovered and exploited.
- **Severity:** The overall criticality based on the above two factors.

Informational

Findings in this category are recommended improvements to enhance code structure, usability, and overall system effectiveness.

Scope

The following was in scope for this audit:

- **Program:** Deadman (Anchor program)
- **Codebase:** `programs/deadman/` (Rust/Anchor)

Findings Summary

Severity	Count
High	1
Medium	1
Low	0

Severity	Count
Informational	3
Total	5

Detailed Findings

[HIGH] Owner-privileged instructions lack time-lock

ID: DEADMAN-001

Contract / File: programs/deadman/src/instructions/owner_withdraw.rs , programs/deadman/src/instructions/cancel_vault.rs , programs/deadman/src/instructions/withdraw.rs

Lines: L35-L46 , L37-L66 , L35-L39

Severity: High

Summary

The beneficiary path correctly enforces the deadman time-lock, but the owner-only paths `owner_withdraw` and `cancel_vault` do not. After the vault has expired, the owner can still withdraw funds or fully cancel the vault, defeating the beneficiary's claim.

Root Cause

`withdraw` computes `unlock_time = last_ping_time + interval + grace_period` and requires the current timestamp to be greater than that threshold before transferring funds to the beneficiary. `owner_withdraw` and `cancel_vault` perform token transfers immediately without any equivalent expiry check.

Impact

Whoever controls the owner key can drain part or all of the vault after the deadman switch has triggered. In the worst case, `cancel_vault` transfers the full balance back to the owner and closes the vault, permanently destroying the beneficiary's ability to claim.

Attack Scenario

1. The vault expires because `last_ping_time + interval + grace_period` has elapsed.
2. The beneficiary prepares to call `withdraw` and claim the tokens.
3. Before or instead of that claim, the owner or an attacker with the owner key calls `owner_withdraw` or `cancel_vault`.
4. Funds are drained or the vault is closed, and the beneficiary loses their intended claim.

Recommendation

In `owner_withdraw` and `cancel_vault`, enforce the same time-lock as in `withdraw`: compute `unlock_time = last_ping_time + interval + grace_period` and **require** that the vault is still active (`clock.unix_timestamp <= unlock_time`) before allowing any owner-driven transfer or close. After expiry, only the beneficiary path (`withdraw`) should be allowed to move or close funds.

Code Snippet

```
// withdraw.rs (correctly time-gated)
let unlock_time = vault_state.last_ping_time
    + vault_state.interval
    + vault_state.grace_period;
require!(clock.unix_timestamp > unlock_time, ErrorCode::VaultStillActive);

// owner_withdraw.rs (no time-lock)
pub fn handler(ctx: Context<OwnerWithdraw>, amount: u64) -> Result<()> {
    token::transfer(transfer_ctx, amount)?;
    Ok(())
}

// cancel_vault.rs (no time-lock)
pub fn handler(ctx: Context<CancelVault>) -> Result<()> {
    token::transfer(transfer_ctx, amount)?;
    token::close_account(close_ctx)?;
    Ok(())
}
```

[MEDIUM] Settings and state can be changed after vault expiry

ID: DEADMAN-002

Contract / File: `programs/deadman/src/instructions/ping.rs`, `programs/deadman/src/instructions/update_beneficiary.rs`, `programs/deadman/src/instructions/update_interval.rs`

Lines: L19-L23, L20-L27, L20-L25

Severity: Medium

Summary

Several owner-controlled state update instructions remain callable even after the deadman switch has expired. This lets the owner reset the timer, change the beneficiary, or modify the interval after the claim should already be fixed.

Root Cause

`ping`, `update_beneficiary`, and `update_interval` do not verify whether the vault has already passed `last_ping_time + interval + grace_period`. The program therefore treats an expired vault as still mutable from the owner's perspective.

Impact

The owner can re-activate an expired vault, redirect the beneficiary, or otherwise grief the intended claimant. This weakens the irrevocability guarantee of the deadman switch.

Attack Scenario

1. The vault expiry time passes, so the beneficiary becomes eligible to withdraw.
2. Before the beneficiary successfully claims, the owner calls `ping`, `update_beneficiary`, or `update_interval`.
3. The timer is reset, the beneficiary is swapped, or the unlock conditions are changed.
4. The beneficiary is locked out, delayed, or redirected from receiving the vault proceeds.

Recommendation

In `ping`, `update_beneficiary`, and `update_interval`, require that the vault has **not** yet expired before applying any state change. At the start of each handler, compute `unlock_time = last_ping_time + interval + grace_period` and require `clock.unix_timestamp <= unlock_time` (e.g. with a dedicated error such as `VaultAlreadyExpired`). Once the vault is past the grace period, treat it as immutable for the owner so only the beneficiary can act via `withdraw`.

Code Snippet

```
// ping.rs
pub fn handler(ctx: Context<Ping>) -> Result<()> {
    let vault_state = &mut ctx.accounts.vault_state;
    let clock = Clock::get()?;
    vault_state.last_ping_time = clock.unix_timestamp;
    Ok(())
}

// update_beneficiary.rs
pub fn handler(ctx: Context<UpdateBeneficiary>, new_beneficiary: Pubkey) -> Result<()> {
    let vault_state = &mut ctx.accounts.vault_state;
    vault_state.beneficiary = new_beneficiary;
    Ok(())
}

// update_interval.rs
pub fn handler(ctx: Context<UpdateInterval>, new_interval: i64) -> Result<()> {
    require!(new_interval > 0, ErrorCode::InvalidInterval);
    require!(new_interval <= MAX_INTERVAL_SECONDS, ErrorCode::IntervalTooLarge);
    let vault_state = &mut ctx.accounts.vault_state;
    vault_state.interval = new_interval;
    Ok(())
}
```

[INFORMATIONAL] Manual account space in `Initialize`

ID: DEADMAN-003

Contract / File: `programs/deadman/src/instructions/initialize.rs`

Lines: L35-L41

Severity: Informational

Summary

The `vault_state` account size is manually calculated inline during initialization instead of being derived from a single canonical constant or `INIT_SPACE`.

Root Cause

The account declaration uses a hardcoded expression for the total serialized size of `VaultState`. This duplicates knowledge of the struct layout in more than one place.

Impact

This is primarily a maintainability and correctness risk. If `VaultState` changes in the future and the manual size expression is not updated, account allocation can become incorrect and cause deployment or runtime issues.

Recommendation

Derive the `vault_state` account size from a single source of truth. For example: define a constant on `VaultState` (e.g. `pub const INIT_SPACE: usize = 8 + 32 + 32 + 32 + 8 + 8 + 8 + 1` or use a helper that sums the serialized size of each field) and use it in the `#[account(..., space = VaultState::INIT_SPACE, ...)]` attribute. When adding or changing fields, update only that constant so account allocation stays correct and the layout is documented in one place.

[INFORMATIONAL] Initialize lacks explicit named minimum bounds

ID: DEADMAN-004

Contract / File: `programs/deadman/src/instructions/initialize.rs`

Lines: L54-L63

Severity: Informational

Summary

The initializer enforces positive values for `interval` and `grace_period`, but does not define explicit named minimum constants that document and centralize the protocol's lower-bound policy.

Root Cause

The code uses direct comparisons (`> 0`) rather than named minimum constants such as `MIN_INTERVAL_SECONDS` and `MIN_GRACE_PERIOD_SECONDS`.

Impact

This does not create an immediate exploit, but it makes policy less explicit and increases the chance of inconsistent validation if bounds are later reused or extended elsewhere in the codebase.

Recommendation

Introduce named minimum constants (e.g. `MIN_INTERVAL_SECONDS` and `MIN_GRACE_PERIOD_SECONDS`) in a shared constants module or at the top of the program, and use them in all validation paths for `interval` and `grace_period`. Replace `interval > 0` with `interval >= MIN_INTERVAL_SECONDS` and `grace_period > 0` with `grace_period >= MIN_GRACE_PERIOD_SECONDS`. This documents the protocol's lower-bound policy in one place and reduces the risk of inconsistent bounds if more instructions touch these values later.

[INFORMATIONAL] Deposits remain allowed after expiry

ID: DEADMAN-005

Contract / File: `programs/deadman/src/instructions/deposit.rs`

Lines: L35-L44

Severity: Informational

Summary

The `deposit` instruction does not check whether the vault has already expired. As a result, behavior depends on whether the beneficiary has already withdrawn and closed the vault accounts.

Root Cause

`deposit` transfers tokens into the vault without validating `clock.unix_timestamp <= last_ping_time + interval + grace_period`.

Impact

This creates inconsistent UX rather than a direct exploit. Deposits may succeed after expiry if the vault still exists, but revert once the beneficiary has already

withdrawn and the accounts have been closed.

Recommendation

In `deposit`, require that the vault is still active before accepting new funds. At the start of the handler, compute `unlock_time = last_ping_time + interval + grace_period` and require `clock.unix_timestamp <= unlock_time`; otherwise return a dedicated error (e.g. `VaultAlreadyExpired` or `DepositAfterExpiry`). This makes behavior consistent (no deposits after expiry), avoids race-dependent outcomes, and gives clients a clear, predictable response when the vault is no longer accepting deposits.